

Entrega 4: Ordenamiento, Problemas NP y Síntesis Fina

- *Explicar el ordenamiento topológico y su utilidad en la planificación de entrenamientos, misiones o evolución de personajes.*
- *Analizar el problema del camino mínimo y su relevancia en la optimización de recursos o desplazamientos en el universo.*
- *Reflexionar sobre la eficiencia de las estructuras y algoritmos implementados a lo largo del trabajo.*

Ordenamiento topológico y su utilidad en la planificación de entrenamientos, misiones o evolución de personajes.

El **ordenamiento topológico** es un algoritmo que se usa en grafos dirigidos donde existen **dependencias** entre tareas.

Sirve para **establecer un orden lógico** en situaciones donde ciertas acciones solo pueden hacerse si otras se completan antes.

En nuestro proyecto lo aplicamos a:

- **Entrenamientos de personajes**, por ejemplo:
 - Resistencia → Velocidad → Vuelo → Combate Aéreo
No se puede entrenar Vuelo sin antes haber aprendido Velocidad.
- **Evolución de habilidades**, por ejemplo:
 - Control de Ki → Fusión
 - Ritmo → Fusión
Fusión solo se habilita cuando ambas condiciones están cumplidas.
- **Planificación de misiones**, por ejemplo:



Una misión puede depender de otra misión previa:

- Recolectar información → Infiltrar la base → Rescatar al rehén.

El ordenamiento topológico sirve para:

- evitar contradicciones en la planificación,
- garantizar que se cumplan todas las dependencias,
- detectar ciclos o errores si hay dependencias imposibles,
- generar automáticamente el orden correcto aunque el usuario cargue todo desordenado.

En nuestra implementación, usamos el **Algoritmo de Kahn**, que funciona con:

- una cola de nodos sin requisitos (grado 0),



Trabajo Final para la Cátedra: Estructuras de Datos en Python

Integrantes: Álvarez Rocío, Varela Ian, Gómez Morena.

- disminución de grados cuando se cumplen dependencias,
- un orden final 100% válido.

Camino mínimo y su relevancia en el universo

El **problema del camino mínimo** consiste en encontrar la ruta más rápida/simple entre dos puntos de un grafo donde las aristas tienen pesos (distancias, costos o riesgos).

En nuestro proyecto modelamos:

- **planetas como nodos,**
- **rutas espaciales como aristas,**
- **distancias en años luz como pesos,**
- **atajos peligrosos como rutas con costo alto.**

Usamos Dijkstra para resolverlo

Dijkstra garantiza encontrar siempre el camino más corto, evaluando progresivamente:

- las rutas conocidas,
- los costos acumulados,
- el camino con menor distancia.

En nuestro universo sirve para optimizar:

- **desplazamientos entre planetas,**
- **consumo de energía de naves,**
- **tiempos de viaje,**
- **recursos de combate,**
- **misiones que deben resolverse rápido.**

Por ejemplo:

- **BFS** cree que lo mejor es Tierra → Portal → Saturno (solo 2 saltos).
- **Dijkstra** descubre que el portal cuesta 1000 años luz, así que prefiere:
Tierra → Marte → Asteroides → Júpiter → Saturno (550 años luz).

Por ende:

- BFS encuentra **menos paradas.**
- Dijkstra encuentra **el camino más rápido.**

Eficiencia de las estructuras y algoritmos implementados a lo largo del trabajo



Trabajo Final para la Cátedra: Estructuras de Datos en Python

Integrantes: Álvarez Rocío, Varela Ian, Gómez Morena.

Durante el trabajo usamos múltiples estructuras y analizamos su rendimiento:

- **Heap binario (cola de prioridades) — Entrega 1**
 - Garantiza siempre obtener la misión más peligrosa primero.
 - Justificación: simulación realista de emergencias tipo Dragon Ball.
- **Listas y CSVs para cargar datos**
 - Suficiente para datasets pequeños/medianos.
- **Árbol binario de poder — Entrega 2**
 - **Árbol general de habilidades**
 - Estructura flexible (muchos hijos por nodo).
 - Recorridos en preorden: **$O(n)$**
 - Ideal para representar evoluciones y transformaciones de las habilidades.
- **Grafos — Entrega 3**
 - BFS y DFS: **$O(V + E)$**
 - DFS para exploración profunda.
 - BFS para encontrar caminos con menos saltos.
- **Dijkstra — Entrega 4**
 - Complejidad: **$O((V + E) \log V)$** usando heap.
 - Permite optimizar rutas reales teniendo en cuenta la distancia.
- **Ordenamiento topológico**
 - Ideal para secuencias de entrenamiento o misiones.

En conclusión, las estructuras elegidas, son apropiadas para el tipo de datos, responden eficientemente incluso en universos grandes y permiten simular, en nuestro caso, situaciones reales de Dragon Ball: misiones, viajes, evolución, entrenamientos, etc.

